SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

urse Code:	CSA0309	**Course Title:**	Data Structures
O Assessed:	CO5 – Naturalization (BL4, BL6)	**Assessment:**	Assessment Tool 2 – Flowchart-To-Code Conversion
eightage:	35% of CIA	**Total Marks:**	100
05 Statement:	*Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.*		
udent Name:	K.Nilavan	**Reg. No.:**	192511028
ction / Batch:	2025 – 2029	**Date:**	02/09/2026

ode of Conduct

K.Nilavan (192511028) *(Name / Reg No)*

ertify that this submission is my original work and that I have adhered to the guidelines specified for this ssessment. I understand that any violation of academic integrity rules will result in disciplinary action.

ignature of the Student: K.Nilavan

nstructions

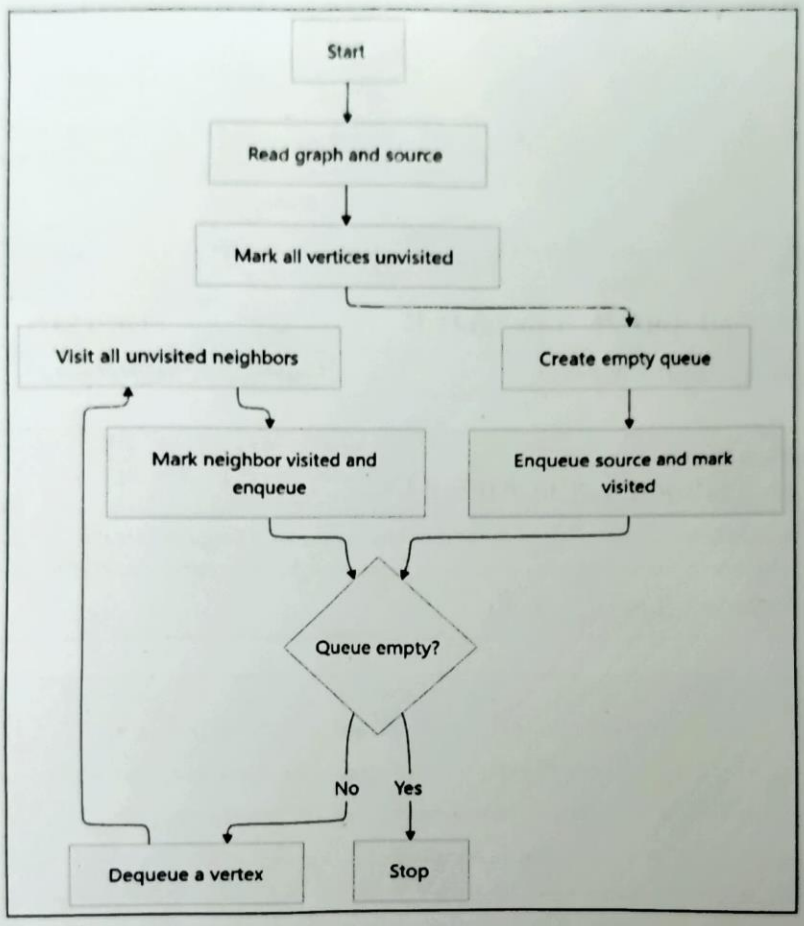
- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
raph Traversal	BFS	1	20
inimum Spanning Tree	Kruskal's Algorithm	2	20
raph Sorting	Topological Sort	3	20
raph Traversal	DFS	4	20
inimum Spanning Tree	Prim's Algorithm	5	20
	GRAND TOTAL:	**100 Marks**	

Annexure A - Flowchart-To-Code Conversion

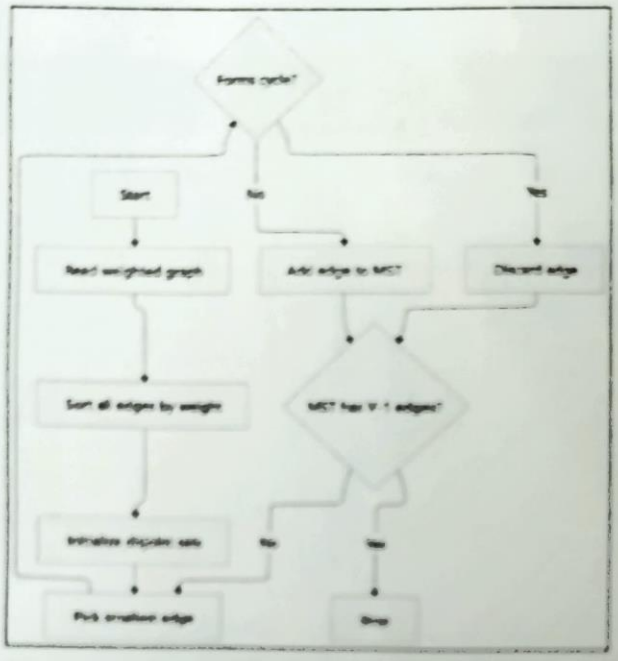
1. Convert the flowchart for Breadth First Search traversal into code. (20 Marks)

Flowchart Diagram:



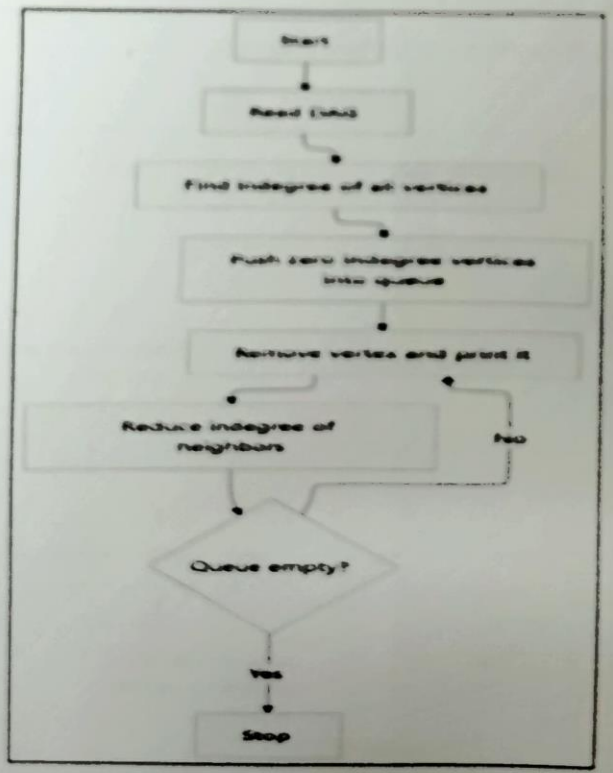
2. Convert the flowchart for Kruskal's algorithm into code. **(20 Marks)**

Flowchart Diagram:



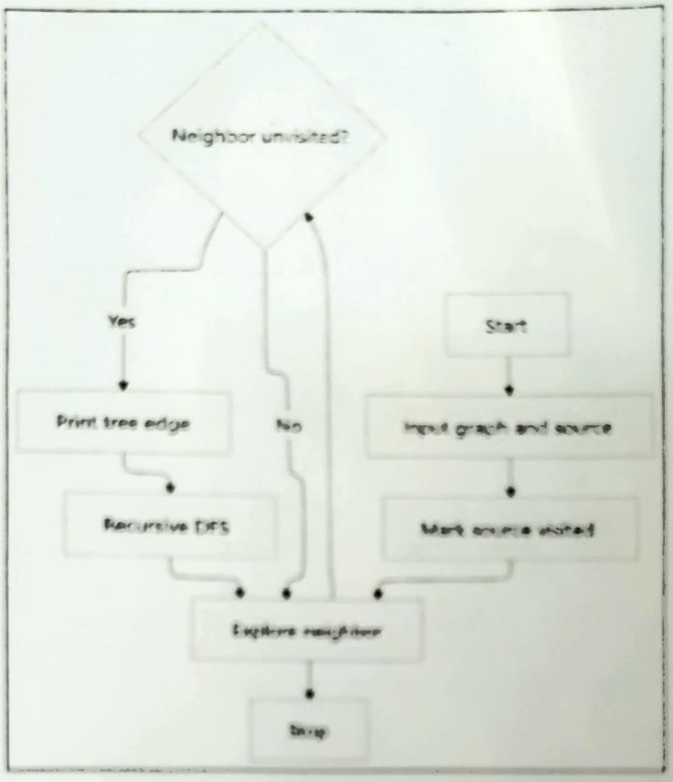
3. Convert the flowchart for topological sort into code. (20 Marks)

Flowchart Diagram:



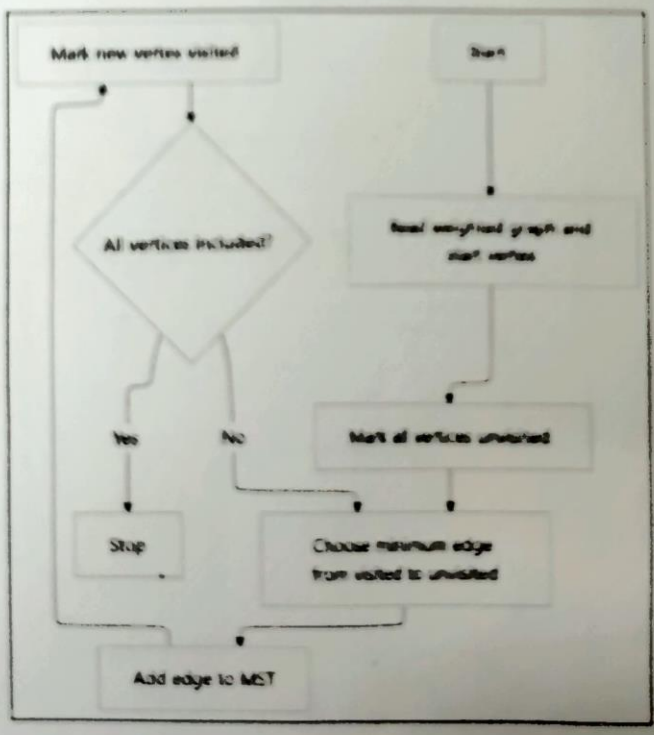
4. Convert the flowchart for generating a DFS tree into code. (20 Marks)

Flowchart Diagram:



5. Convert the flowchart for Prim's algorithm into code. (20 Marks)

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Breadth First search:

```
#include <stdio.h>
int main()
{
    int g[5][5] = { {0,1,0,0,0}, {1,0,0,1,0}, {1,0,0,0,1},
                    {0,1,0,0,1}, {0,0,1,1,0} };
};
int q[5], v[5] = {0}, f=0, r=0, i, x;

q[r++] = 0;
v[0] = 1;
while (f<r)
{
    x = q[f++];
    printf("%d", x);
    for(i=0; i<5; i++)
        if(g[x][i] && !v[i])
        {
            v[i] = 1;
            q[r++] = i;
        }
  }
  return 0;
}
```

2) ## Kruskal's Algorithm:

```
#include <stdio.h>
int p[5];
int find(int x)
```

```
{
 while (p[x] != x)
      x = p[x];
 return x;
}
int main()
{   int e[][3] = {{1,2,1},{1,3,2},{3,4,2},{0,2,3},{0,1,4}};
    int i, a, b, c, t = 0;
    for (i=0; i<5; i++) p[i]=i;
    for (i=0; i<5; i++)
    {
       a = find (e[i][0]);
       b = find (e[i][1]);
       if (a != b)
       {
           print ("%d - %d = %d\n", e[i][0], e[i][1], e[i][2]);
           p[a]=b;
           t += e[i][2];
       }
    }
    printf("Total = %d", t);
    return 0;
}
```

3) Topological Sort:

```
# include <stdio.h>
int main()
{
   int g[6][6] = {{0,1,1,0,0,0},{0,0,0,1,0,0},{0,0,0,1,1,0}
```

```
        {0,0,0,0,0,1},{0,0,0,0,0,0}
};
int in[6]={0}, q[6], f=0, r=0, i, j, x;
for(i=0; i<6; i++)
    for(j=0; j<6; j++)
        if(g[i][j]) in[j]++;
for(i=0; i<6; i++)
    if(in[i]==0) q[r++]=i;
while(f<r)
{
    x=q[f++];
    printf("%d", x);
    for(i=0; i<6; i++)
        if(g[x][i] && --in[i]==0)
            q[r++]=i;
}
return 0;
}
```

H) DFS Tree:

```
#include <stdio.h>
int g[5][5]={{0,1,1,0,0},{1,0,0,1,0},{1,0,0,0,1},{0,1,0,0,1}
        ,{0,0,1,1,0}
};
int v[5];
void dfs(int x)
{
    int i;
```

```
    v[x] = 1;
    printf("%d", x);

    for(i=0; i<5; i++)
        if(g[x][i] && !v[i])
            dfs(i);
}
int main()
{
    dfs(0);
    return 0;
}
```

5) Prim's Algorithm:

```
#include<stdio.h>
int main()
{ int g[5][5] = {{0,4,3,0,0},{4,0,1,2,0},{3,1,0,4,5},
                 {0,2,4,0,2},{0,0,5,2,0}
};
int v[5]={1,0,0,0,0}, n=0, t=0;
int i, j, min, a, b;
while (n<5)
{
    min = 999;
    for(i=0; i<5; i++)
      if(v[i])
        for(j=0; j<5; j++)
          if[!v[j] && g[i][j] && g[i][j]<min)
          {   min = g[i][j];
              a=i;
              b=j;
          }
```

```
        printf("%d - %d = %d\n", a, b, min);
        t += min;
        v[b] = 1;
        n++;
    }
    printf("Total = %d", t);
    return 0;
}
```